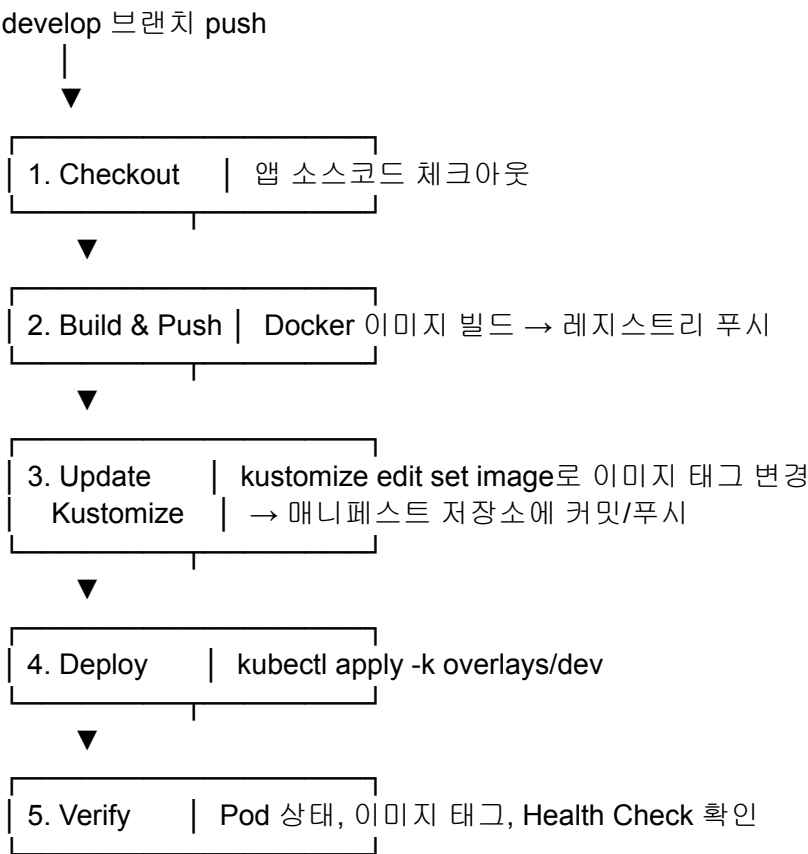


Jenkins + Kustomize 개발계 배포 파이프라인 가이드

1. 파이프라인 전체 흐름



2. 사전 요구사항

Jenkins Credentials 등록

Credentials ID	종류	용도
harbor-credentials	Username/Password	컨테이너 레지스트리(Harbor) 인증

kubeconfig-dev	Secret file	개발 클러스터 kubeconfig
gitlab-token	Secret text	GitLab API 토큰 (매니페스트 저장소 push용)

Jenkins 에이전트 필수 도구

- docker (이미지 빌드)
- kubectl (클러스터 배포)
- kustomize (매니페스트 변경)
- git (매니페스트 저장소 클론/푸시)

3. Jenkinsfile 전문

```

pipeline {
  agent {
    kubernetes {
      yaml '''
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: builder
      image: docker:24-dind
      securityContext:
        privileged: true
      volumeMounts:
        - name: docker-sock
          mountPath: /var/run/docker.sock
    - name: deployer
      image: bitnami/kubectl:1.28
      command: ['sleep']
      args: ['infinity']
  volumes:
    - name: docker-sock
      hostPath:
        path: /var/run/docker.sock
'''
    }
  }
}
```

```

// -----
// 환경 변수
// -----
environment {
    // 레지스트리
    REGISTRY      = 'harbor.example.com'
    IMAGE_REPO    = "${REGISTRY}/myproject/my-app"
    IMAGE_TAG     = "dev-${GIT_COMMIT[0..6]}-${BUILD_NUMBER}"

    // Git / Kustomize
    KUSTOMIZE_REPO = 'https://gitlab.example.com/infra/k8s-manifests.git'
    KUSTOMIZE_BRANCH = 'main'
    OVERLAY_PATH   = 'k8s/overlays/dev'

    // 배포 대상
    NAMESPACE     = 'dev'
    APP_NAME      = 'my-app'
}

options {
    timeout(time: 15, unit: 'MINUTES')
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '20'))
    timestamps()
}

triggers {
    gitlab(
        triggerOnPush: true,
        branchFilterType: 'NameBasedFilter',
        includeBranchesSpec: 'develop'
    )
}

stages {

    // -----
    // 1단계: 체크아웃
    // -----
    stage('Checkout App Source') {
        steps {
            checkout scm
            script {
                env.GIT_SHORT_COMMIT = sh(

```

```

        script: 'git rev-parse --short=7 HEAD',
        returnStdout: true
    ).trim()
    env.IMAGE_TAG = "dev-${env.GIT_SHORT_COMMIT}-${BUILD_NUMBER}"
    echo "빌드 이미지 태그: ${env.IMAGE_TAG}"
}
}
}

```

```

// -----
// 2단계: Docker 이미지 빌드 & 푸시
// -----

```

```

stage('Build & Push Image') {
    steps {
        container('builder') {
            withCredentials([
                usernamePassword(
                    credentialsId: 'harbor-credentials',
                    usernameVariable: 'REG_USER',
                    passwordVariable: 'REG_PASS'
                )
            ]) {
                sh """
                    echo "${REG_PASS}" | docker login ${REGISTRY} \
                        -u "${REG_USER}" --password-stdin

                    docker build -t ${IMAGE_REPO}:${IMAGE_TAG} .
                    docker push ${IMAGE_REPO}:${IMAGE_TAG}
                    docker logout ${REGISTRY}
                """
            }
        }
    }
}

```

```

// -----
// 3단계: Kustomize 매니페스트 업데이트
// -----

```

```

stage('Update Kustomize Manifest') {
    steps {
        container('deployer') {
            withCredentials([
                string(credentialsId: 'gitlab-token', variable: 'GIT_TOKEN')
            ]) {

```

```

sh """
    # k8s 매니페스트 저장소 클론
    git clone
https://oauth2:\${GIT_TOKEN}@gitlab.example.com/infra/k8s-manifests.git
    cd k8s-manifests/\${OVERLAY_PATH}

    # kustomize로 이미지 태그 변경
    # → base yaml은 건드리지 않음
    # → overlay의 kustomization.yaml만 수정
    kustomize edit set image \${IMAGE_REPO}:\${IMAGE_TAG}

    # 변경사항 확인
    echo "=== kustomization.yaml 변경내용 ==="
    cat kustomization.yaml

    echo ""
    echo "=== 최종 빌드 결과 미리보기 ==="
    kustomize build . | head -80

    # 변경사항 커밋 & 푸시
    git config user.email "jenkins@example.com"
    git config user.name "Jenkins CI"
    git add kustomization.yaml
    git commit -m "[DEV] Update image tag: \${IMAGE_TAG}"

    App commit: \${GIT_SHORT_COMMIT}
    Jenkins build: #\${BUILD_NUMBER}"

    git push origin \${KUSTOMIZE_BRANCH}
    """
}
}
}
}

// _____
// 4단계: 개발 클러스터에 배포
// _____
stage('Deploy to Dev') {
    steps {
        container('deployer') {
            withCredentials([
                file(credentialsId: 'kubeconfig-dev', variable: 'KUBECONFIG')
            ]) {

```

```

sh """
echo "=== 배포 전 diff 확인 ==="
kubectl diff -k k8s-manifests/${OVERLAY_PATH} || true

echo ""
echo "=== 개발 환경 배포 시작 ==="
kubectl apply -k k8s-manifests/${OVERLAY_PATH}

echo ""
echo "=== 롤아웃 상태 확인 ==="
kubectl -n ${NAMESPACE} rollout status \
    deployment/${APP_NAME} --timeout=300s
"""
}
}
}
}

// -----
// 5단계: 배포 검증
// -----
stage('Verify Deployment') {
    steps {
        container('deployer') {
            withCredentials([
                file(credentialsId: 'kubeconfig-dev', variable: 'KUBECONFIG')
            ]) {
                sh """
                echo "=== Pod 상태 확인 ==="
                kubectl -n ${NAMESPACE} get pods \
                    -l app=${APP_NAME} -o wide

                echo ""
                echo "=== 배포된 이미지 확인 ==="
                DEPLOYED_IMAGE=$(kubectl -n ${NAMESPACE} \
                    get deployment/${APP_NAME} \
                    -o jsonpath='{.spec.template.spec.containers[0].image}')
                echo "배포된 이미지: ${DEPLOYED_IMAGE}"

                if echo "${DEPLOYED_IMAGE}" | grep -q "${IMAGE_TAG}"; then
                    echo "✅ 이미지 태그 일치 확인 완료"
                else
                    echo "❌ 이미지 태그 불일치!"
                    exit 1
                fi
            }
        }
    }
}

```

```

        fi

        echo ""
        echo "=== Health Check ==="
        kubectl -n ${NAMESPACE} exec \
            deployment/${APP_NAME} -- \
            wget -qO- http://localhost:8080/health || \
            echo "⚠️ Health check 미응답 (확인 필요)"
    ""
}
}
}
}

// -----
// 후처리
// -----
post {
    success {
        echo ""
        =====
        ✅ 개발계 배포 완료
        =====

        앱:      ${APP_NAME}
        이미지:  ${IMAGE_REPO}:${IMAGE_TAG}
        네임스페이스: ${NAMESPACE}
        커밋:    ${GIT_SHORT_COMMIT}
        =====
        ""

        // Slack 알림 (선택)
        // slackSend(
        //   channel: '#dev-deploy',
        //   color: 'good',
        //   message: "✅ [DEV] ${APP_NAME} 배포 완료 - ${IMAGE_TAG}"
        // )
    }

    failure {
        echo "❌ 배포 실패 - 로그를 확인하세요"
        // slackSend(
        //   channel: '#dev-deploy',
        //   color: 'danger',
        //   message: "❌ [DEV] ${APP_NAME} 배포 실패 - Build #${BUILD_NUMBER}"
        // )
    }
}

```

5. VM 기반 **Jenkins** 에이전트 사용 시

위 샘플은 Kubernetes Pod 에이전트를 사용합니다. 기존 VM 기반 에이전트를 사용한다면 아래와 같이 변경하세요.

// 변경 전: Kubernetes Pod 에이전트

```
agent {
  kubernetes {
    yaml "..."/>
  }
}
```

// 변경 후: VM 에이전트

```
agent {
  label 'your-jenkins-agent'
}
```

VM 에이전트 사용 시 `container('builder')`, `container('deployer')` 블록을 제거하고 `steps` 안에 직접 명령어를 작성하면 됩니다.

// 변경 전

```
stage('Deploy to Dev') {
  steps {
    container('deployer') {
      withCredentials([...]) {
        sh "kubectl apply -k ..."
      }
    }
  }
}
```

// 변경 후

```
stage('Deploy to Dev') {
  steps {
    withCredentials([...]) {
      sh "kubectl apply -k ..."
    }
  }
}
```

6. 환경에 맞게 수정할 항목 체크리스트

항목	현재 샘플 값	확인
레지스트리 주소	harbor.example.com	☐
이미지 경로	myproject/my-app	☐
매니페스트 저장소 URL	gitlab.example.com/infra/k8s-manifests.git	☐
overlay 경로	k8s/overlays/dev	☐
네임스페이스	dev	☐
앱 이름	my-app	☐
Health Check 경로	/health	☐
Credentials ID 3개	harbor-credentials, kubecfg-dev, gitlab-token	☐
Jenkins 에이전트 타입	Kubernetes Pod / VM 중 선택	☐
Slack 알림 채널	#dev-deploy (주석 해제 시)	☐